



Detecta, Decide y Refactoriza

Actividad de Clase

Nombre del alumno:

Fecha:

ANGEL DAVID ECHENIQUE VILLANUEVA

25/02/26

Instrucciones

Ejercicio diseñado para principiantes basado en el módulo Refactorización y Mejora de Código. Responde cada sección con lo aprendido en clase.



PARTE1

Parte Teórica: Comprender la Refactorización

Responde brevemente a las siguientes preguntas con tus propias palabras:

1.1 ¿Por qué una función debe tener una sola responsabilidad?

Tu respuesta:

Porque es mas facil de entender y mantener. Cuando hace muchas cosas una funcion puede ser propensa a errores.

1.2 ¿Qué es un valor mágico y por qué debe evitarse?

Tu respuesta:

Puede ser un numero o texto que aparece dentro del codigo sin explicacion de la nada.
Porque evitarla, hace que el codigo sea dificil de de entender y mantener. Si es posible cambialo por conts de nombres descriptivos para mejor legibilidad.

1.3 ¿Qué diferencia hay entre simplificar y optimizar código?

Tu respuesta:

Simplificar es hacer el codigo mas legible al ojo humano.
Optimizar es buscar que el codigo tenga un rendimiento mejor.

“La refactorización mejora la estructura interna del código sin modificar su comportamiento externo.”



Ejercicio Práctico: Detectar Code Smells

Observa este código con atención:

```
def procesar(lista):  
    r = 0    for x in  
lista:      r = r + x *  
2          if r > 200:  
r = r - 15  
print("Resultado:", r)
```

2.1 Identifica tres problemas del código

Escribe qué problema detectas en cada punto:

Nº	Problema detectado	Por qué es un problema
1	R / x / Lista	Legibilidad muy compleja no hay descriptividad
2	Valor magico 200 y 15	Son numeros sin contexto
3	La funcion hace muchas cosas a la ves	Calcula y ademas imprime mezcla logica con presentacion.

2.2 ¿Cómo lo mejorarías?

Solo explicación, sin reescribir el código:

Tu propuesta de mejora:

Primero que nada renombraría las funciones y las variable. Separaría responsabilidades porque así es mejor para mantenerlo, una función para cálculo y otra para mostrar el resultado eliminando valores mágicos y poniendo constante con nombres significativos.

“Renombra variables y funciones siguiendo convenciones de Clean Code.”



PARTE3

Mini-Caso: Decide el Tipo de Refactorización

Elige el tipo de refactorización más adecuado para cada situación:

Nº	Situación	Tipo de refactorización
1	Una función tiene 8 condicionales anidados.	Simplificar condicionales
2	Un archivo tiene 3 funciones que hacen casi lo mismo.	Extract method
3	Un algoritmo tarda demasiado porque usa dos bucles anidados.	Optimizacion de algoritmos
4	El código usa nombres como a, b, fn1.	Renombramiento de variable
5	El proyecto usa callbacks antiguos en JavaScript.	Aplicacion de async-await

“La IA puede realizar distintos tipos de refactorización según el objetivo.”



PARTE4

Actividad de Investigación Guiada

Investiga por tu cuenta y responde:

4.1 ¿Qué es la complejidad ciclomática?

Tu respuesta:

Por lo que investigue, es una metrica que mide cuantos caminos logicos diferentes existen dentro de un programa o funcion.

Un valor alto indica que el codigo es mas dificil de probar mantener y entender.

4.2 ¿Qué patrón de diseño te parece más fácil de entender y por qué?

Tu respuesta:

El patron Singleton porque por su diseño asegura que una clase tenga una unica instancia y que sea accesible desde cualquier parte del programa.

4.3 ¿Qué significa que un código tenga alto acoplamiento?

Tu respuesta:

Que los modulos, clases o funciones despende uno del otro lo cual vuelve al codigo fragil antes cualquier cambio.

"Alta cohesión, bajo acoplamiento."



PARTE5

Actividad Final: Crear un Prompt de Refactorización

A partir de este código:

```
def total(a, b):  
    t = a * b    if a  
> 10:          t = t  
    * 0.9       return t
```

5.1 Escribe un prompt completo para pedir a la IA que refactorice este código

Mi prompt completo:

Necesito que me refactorices este codigo aplicando clean code, cambiando las variables y funciones por nombres descriptivo tambien eliminame esos valores magicos por consts.

5.2 ¿Por qué tu prompt es mejor que uno simple como "Refactoriza este código"?

Mi explicación:

Porque yo estoy guiando a la IA, no la IA a mi. Aparte estoy dandole parametros a seguir.

"Prompt típico: 'Simplifica esta función eliminando anidación innecesaria.'"

Entrega

Incluye en tu entrega:

1. Respuestas teóricas (Parte 1)
2. Detección de problemas del código (Parte 2)
3. Elección de tipos de refactorización (Parte 3)
4. Investigación personal (Parte 4)
5. Prompt final explicado (Parte 5)